

C++_QT_3일차 과제2, 과제3 구현 보고서

이름: 김도훈
학번: 2026406042
학과: 로봇학부

<목차>

- 1. 과제2
 - 1_1 과제2 설명
 - 1_2 구현
 - 1_3 실행 결과
- 2. 과제3
 - 2_1 과제3 설명
 - 2-2 구현
 - 2_3 실행 결과

1. 과제2

1_1 과제2 설명

과제2는 A* 알고리즘과 다익스트라 알고리즘을 QT를 사용해서 미로에서 시작점과 도착점의 최단 거리를 찾는 것을 GUI로 시각화하는 것이다. 그리고 시각화하는 과정에서 각 알고리즘이 탐색한 노드 수, 최종 경로 비용, 실행 시간을 GUI를 활용해 출력하여 비교하는 프로그램이다. 추가적으로 각 알고리즘이 사용될 미로 맵에는 수동으로 장애물, 시작점, 도착점, 크기를 설정할 수 있도록 구현하는 것이 과제2의 목표이다.

1_2 과제2 구현

UI 배치

UI 배치는 미로 맵 두 개로 사용할 위젯 2개, 장애물 설정, 도착점, 시작점, 크기 설정 버튼, 알고리즘 실행 버튼(푸쉬 버튼 5개) 알고리즘의 탐색 노드 수, 최종 경로 비용, 시간 등을 출력할 linedit 6개를 출력 창에 QT Designer에서 배치했다. ui 배치에서 가장 중요한 것은 위젯을 미로 맵으로 사용한 것이다. 위젯은 그냥 빈 사각형이지만 본 프로그램에서는 위젯에 미로, 장애물, 시작점, 도착점 등을 구현했다. 이 과정을 위해서는 MazeWidget이라는 클래스로 상속하여 위젯을 커스텀하여 사용할 수 있도록 했다.

MazeWidget

Mazewidget 클래스는 미로 출력창에 그려주는 역할을 한다고 생각하면 된다. 이 클래스에는 미로를 화면에 표현하고 사용자가 직접 미로를 편집할 수 있도록 만든 커스텀 QWidget이다. setMaze, setMapData, resizeMaze를 통해 미로 데이터를 관리하고 setMode와 mousePressEvent를 이용해 장애물, 시작점, 도착점을 사용자가 설정할 수 있도록 구현했다. 또한 setVisited와 setPath를 통해 알고리즘의 탐색 결과를 전달받고 paintEvent에서 방문한 노드와 최단 경로를 색으로 표현하도록 구현했다. 마지막으로 mazeEdited signal을 이용해

왼쪽 미로가 수정되면 오른쪽 미로에도 동일하게 반영되도록 했다.

다익스트라 알고리즘

다익스트라 알고리즘은 시작점에서 각 노드까지의 최단 거리를 계산하는 알고리즘이다. 본 프로그램에서는 각 미로 칸의 이동 비용을 모두 1로 설정하였다. 각 노드까지의 최소 비용은 다음 변수에 저장한다.

```
std::vector<std::vector<int>>> dist;
```

처음에는 모든 노드의 거리를 매우 큰 값인 INT_MAX로 설정하고 시작점의 비용만 0으로 설정하였다.

```
dist[start.first][start.second] = 0;
```

다익스트라 알고리즘의 핵심은 아직 방문하지 않은 노드 중 현재까지의 비용이 가장 작은 노드를 선택하는 것이다.

```
if ( !visited[x][y] && ( ux == -1 || dist[x][y] < dist[ux][uy] ))
```

선택된 노드는 방문 처리하고 탐색한 노드 수를 1 증가시킨다.

```
visited[ux][uy] = true;  
exploredCount++;
```

그 후 현재 위치를 기준으로 상하좌우 네 방향을 확인한다.

```
static int dx[4] = { -1, 1, 0, 0 };  
static int dy[4] = { 0, 0, -1, 1 };
```

이동할 위치가 장애물이 아니고 기존에 저장된 거리보다 새로운 거리가 더 짧은 경우 거리 값을 갱신한다.

```
if ( dist[ux][uy] + 1 < dist[nx][ny] ){  
    dist[nx][ny] = dist[ux][uy] + 1;  
    parent[nx][ny] = {ux, uy};  
}
```

parent에는 현재 노드가 어떤 노드에서 왔는지를 저장한다. 이후 도착점까지 탐색이 완료되면 parent를 거꾸로 따라가 최종 경로를 구한다.

A* 알고리즘

A* 알고리즘은 다익스트라 알고리즘과 비슷하지만 목적지까지의 예상 거리도 함께 고려한다. 다익스트라는 현재까지 실제로 이동한 비용만 사용하지만 A*는 다음 계산식을 사용한다.

$$f = g + h$$

여기서 g 는 시작점부터 현재 위치까지 실제 이동한 비용이고, h 는 현재 위치에서 도착점까지의 예상 거리이다.

본 프로그램에서는 상하좌우로만 이동하기 때문에 휴리스틱 값으로 맨해튼 거리를 사용하였다.

```
std::abs(x - end_.first) + std::abs(y - end_.second);
```

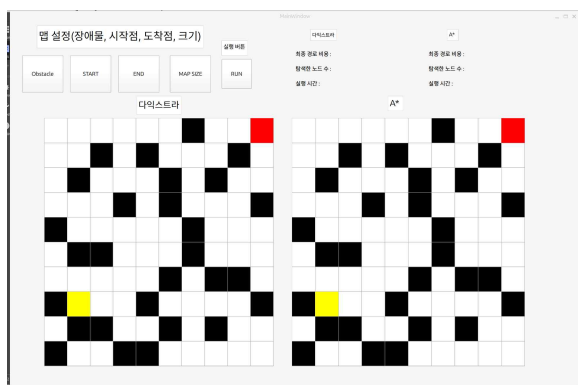
예를 들어 현재 위치가 (2, 3)이고 도착점이 (7, 8)이면 예상 거리는 다음과 같이 계산된다.

$|2 - 7| + |3 - 8| = 5 + 5 = 10$ A* 알고리즘에서는 방문하지 않은 노드 중 $g + h$ 값이 가장 작은 노드를 선택한다.

```
int fCost = gCost[x][y] + heuristic(x, y);
```

이 방식으로 목적지에 가까운 방향을 우선적으로 탐색하기 때문에 일반적으로 다익스트라 알고리즘보다 적은 노드를 탐색할 수 있다.

1.3 실행 결과



2. 과제3

2.1 과제3 설명

과제3은 QT를 활용해 천지인 키보드를 구현하는 것이다. 과제3은 다 기능을 가진 프로그램을 QT로 제작해보면서 QT의 많은 기능들을 사용해보고 익숙해지는 것이 목표이다.

2_2 과제3 구현

<구현한 기능>

1. 영어 입력 기능(기본 상태)

코드를 실행시키면 가장 기본으로 나오는 상태가 이 기능이다. 키보드는 영어 소문자로 되었다. 이 키보드를 누를 시 영어 소문자가 라벨에 입력된다.

2. 영어 글자 자동 확장

천지인 키보드에는 키 하나가 문자 3개 이상을 출력해야한다. 이 기능을 QTimer를 사용해 키를 한 번 누르고 1초 뒤에 키의 두 번째 문자출력을 하고 또 1초 뒤에 세 번째 문자를 출력하도록 해 구현하였다

3. 영어 소문자 키보드 + shift => 영어 대문자

기본 상태가 영어 소문자이기 때문에 shift를 누르면 영어 대문자 모드로 변경되고 대문자가 출력되도록 해당하는 버튼 함수에 이 기능을 구현하였다.

4. 한/영 전환 키

이것 또한 3번과 같은 방식으로 구현을 했고 예시 천지인 키보드와 동일하게 하나의 키로 한글 모드 영어 모드 전환이 가능하도록 구현했다.

5. 한글 입력 기능

6. 숫자/특수문자 모드 전환 기능, 숫자 입력 기능

이것은 3번과 완전히 같은 기능이다. 출력만 다르게하여 숫자와 특수문자에 해당하는 버튼으로 변경되게 하고 숫자와 특수 문자를 출력하도록 했다.

7. 삭제 기능

삭제 기능은 한 문자가 lineedit에 출력된 한 문자를 삭제해야한다. 그래서 backspace() 함수를 사용해 커서 앞의 문자가 지워지도록 하였다.

8. 스페이스 기능

스페이스 기능은 띄어쓰기가 되어야 하기 때문에 이 버튼을 클릭 시 공백을 입력하면서 띄어쓰기를 구현하였다.

9. enter 기능

enter 기능 문자 입력을 끝냈을 때 사용하는 키이다. enter 키를 눌렀을 때 파일 입출력을 사용하여 파일에 lineedit에 입력한 텍스트를 출력하도록 했다. 또한 enter 누를 시 입력한 문자는 모두 삭제된다.

이 뿐만 아니라 한글 출력을 완벽하게 구현하기 위해서

1. 아래아와 모음을 사용하여 다른 모음을 조합하여 사용하는 기능

2. 한글은 초성 중성 종성이 한 문자가 되는 것을 구현하기 위해 초성 중성 종성 저장 기능

3. 저장한 초성 중성 종성 조합하여 완성형 문자 만들기 기능

4. 쌍자음 입력 기능

5. 겹받침 생성 기능(분리 포함)

6. 받침 이동 기능 ex) 간 + ㅣ = 가나

이러한 기능들을 구현하여 천지인 키보드를 구현하였다. 핵심 기능들만 자세한 구현 방법을 설명하겠다.

ui배치

ui배치는 천지인 키보드와 똑같이 배치했다. 푸쉬 버튼 17개를 그리드 레이아웃을 사용하여 출력창에 키보드를 구현했다. 라벨을 사용해서 천지인 키보드로 누른 문자를 출력이 되도록 하였다. 다음 모든 버튼의 go to slot을 하여 버튼이 눌렸을 때 해당 버튼의 함수가 실행되게 하였다.

먼저 영어 출력을 구현했다. 한 키에 세 개 정도의 문자를 출력해야했기 때문에

inputEnglishKey() 함수를 사용해서 한 키에서 세 개의 문자가 출력되도록 했다.

기본 키보드 상태는 영어 소문자 상태인데 이때 shift키를 누르면 영어 대문자 상태로 바뀐다. 이것을 구현하기 위해 shift 키에 해당하는 함수에 버튼 위 텍스트를 바꿔게 만들었다. 코드는 다음과 같다. ui->btn버튼명->setText("바꿀 텍스트"); 이렇게 버튼 위에 텍스트를 변경했고 실제로 버튼을 눌렀을 때도 해당하는 문자가 출력되도록 해야한다. 이것을 위해서 쉬프트가 눌렸을 때를 확인하기 위해서 bool isUpperCase = false; bool 자료형으로 쉬프트 연산키 눌림 여부를 확인할 수 있는 변수를 만들었다. false면 버튼이 눌린 것이 아님 true면 버튼이 눌림. 이 방식은 한영키 전환, 숫자 키 전환 방식에도 사용했다.

```
void MainWindow::on_btnMNO_clicked()
{
    if (Korean)
        inputKoreanConsonant(ui->btnMNO, "ㄱ ㅋ ㆁ");

    else if(Number)
        inputEnglishKey(ui->btnMNO, "6<=>");

    else
        inputEnglishKey(ui->btnMNO, "mno");
}
```

위의 코드처럼 조건문을 사용해서 조건 3가지에 따라 버튼을 눌렀을 때 출력되는 문자가 다르게 구현하였다. 이 방식으로 숫자, 영어 소문자, 대문자, 특수문자 등 대부분의 문자를 출력할 수 있었다. 하지만 문제는 한글 출력이다. 영어 출력은 입력한대로 그대로 출력하면 되지만 한글은 초성 중성 종성을 조합해서 하나의 문자를 만든다. 그래서 영어 출력보다 더 어렵다.

한글 출력

한글 출력을 위해서 QString koreanInitial;, QString koreanVowel;, QString koreanFinal; 세 가지의 변수가 사용된다. 순서대로 초성 중성 종성을 저장하는 변수이다. 한글을 출력할 때

이 세 가지 변수 중 두 가지 이상의 변수를 사용해서 하나의 문자를 만든다. 또한 천지인 키보드에서 모음을 만들 때 아래아와 모음을 조합해 새로운 모음을 만들어 낸다. 예를 들어 $\cdot + | = ㅓ$, $\cdot + - = ㅕ$ 이다. 본 프로그램에서는 cheonjiinBuffer 라는 문자 배열을 만들어 누른 순서대로 해당하는 모음이나 아래아를 저장한 후 다시 누른 순서대로 꺼내 그 규칙대로 중성을 판별하여 출력한다. 이 규칙이 매우 많아서 한글 출력 코드에 대부분이 이 코드이다. 코드는 다음과 같다.

```
if (buffer == " | \cdot ") return "ㅓ";
// ㅓ
if (buffer == " | \cdot | ") return "ㅕ";
// ㅕ
if (buffer == " | \cdot \cdot ") return "ㅗ";
// ㅗ
```

이것은 중성을 출력할 때 판별하는 코드이다. 최종적으로 한글 완성형 한 문자를 출력하기 위해서는 유니코드 계산식을 사용해서 한 글자를 만들어 출력했다. 코드에 사용된 유니코드 계산식은 다음과 같다.

```
int unicode = 0xAC00 + (initialIndex * 21 + vowelIndex) * 28 + finalIndex;
```

이 계산식은 combineKorean() 함수 초성, 중성, 종성이 몇 번째 인지 찾은 뒤 하나의 글자를 생성한다. 본 프로그램에서는 여기서 생성된 글자를 완성형 한 문자를 출력할 때 사용한다.

2.3 실행 결과

